

the placement of blank space within a single line of code, including the indentation at the front of the line.

Use Space Characters for Indentation

Indentation is the whitespace at the beginning of a code line. You can use one of two whitespace characters, a space or a tab, to indent your code. Although either character works, the best practice is to use spaces instead of tabs for indentation.

The reason is that these two characters behave differently. A space character is always rendered on the screen as a string value with a single space, like this ' '. But a tab character, which is rendered as a string value containing an escape character, or '\t', is more ambiguous. Tabs often, but not always, render as a variable amount of spacing so the following text begins at the next tab stop. The tab stop are positioned every eight spaces across the width of a text file. You can see this variation in the following interactive shell example, which first separates words with space characters and then with tab characters:

```
>>> print('Hello there, friend!\nHow are you?')
Hello there, friend!
How are you?
>>> print('Hello\tthere,\tfriend!\nHow\tare\tyou?')
Hello   there,   friend!
How     are     you?
```

Because tabs represent a varying width of whitespace, you should avoid using them in your source code. Most code editors and IDEs will automatically insert four or eight space characters when you press the TAB key instead of one tab character.

You also can't use tabs *and* spaces for indentation in the same block of code. Using both for indentation was such a source of tedious bugs in earlier Python programs that Python 3 won't even run code indented like this; it raises a `TabError: inconsistent use of tabs and spaces in indentation` exception instead. Black automatically converts any tab characters you use for indentation into four space characters.